

Website Health Check - Alert Groups Management Guide

Overview

The alert groups system provides centralized notification management for your website health monitoring. Instead of managing individual email addresses for each URL, you create alert groups and assign URLs to these groups. This makes it easy to update notification lists without touching individual URL configurations.

Alert Groups Structure

1. Alert Groups (`alert_groups`)

- **Purpose:** Define logical groups of people who should receive alerts
- **Examples:** critical-ops, dev-team, business-hours, executives

2. Alert Group Members (`alert_group_members`)

- **Purpose:** Define who belongs to each group and how to contact them
- **Notification Types:** email, SMS, telegram, webhook
- **Examples:** Individual email addresses, SMS numbers, Telegram chat IDs

3. URL-Alert Group Links (`url_alert_groups`)

- **Purpose:** Connect URLs to alert groups with specific alert type preferences
- **Flexibility:** Control which alert types trigger notifications (down, up, timeout, error)

Setup Workflows

1. Database Setup Workflow

- **Name:** "Health Check - Database Setup"
- **Purpose:** Creates all tables, indexes, procedures, and sample data
- **Run:** Once during initial setup (Manual trigger)

2. Main Monitoring Workflow

- **Name:** "Website Health Check Monitor"
- **Purpose:** Performs health checks and sends alerts via alert groups
- **Run:** Every minute (Schedule trigger)

3. Daily Cleanup Workflow

- **Name:** "Health Check - Daily Cleanup"
- **Purpose:** Removes old data and optimizes database
- **Run:** Daily at 2 AM (Schedule trigger)

Managing Alert Groups

Create a New Alert Group

sql

```
INSERT INTO alert_groups (group_name, description)
VALUES ('mobile-team', 'Mobile development team alerts');
```

Add Members to Alert Group

sql

```
-- Add email member
INSERT INTO alert_group_members (group_id, notification_type, destination, member_name)
VALUES (5, 'email', 'mobile-dev@company.com', 'Mobile Dev Team');

-- Add SMS member
INSERT INTO alert_group_members (group_id, notification_type, destination, member_name)
VALUES (5, 'sms', '5551234567@vztext.com', 'Mobile Lead');
```



Link URL to Alert Group

sql

```
INSERT INTO url_alert_groups (url_id, group_id, alert_on_down, alert_on_up, alert_on_t)
VALUES (1, 5, TRUE, TRUE, TRUE);
```



Update Alert Group Members

sql

-- Update email address

```
UPDATE alert_group_members
SET destination = 'new-email@company.com'
WHERE group_id = 5 AND destination = 'old-email@company.com';
```

-- Disable member temporarily

```
UPDATE alert_group_members
SET is_active = FALSE
WHERE group_id = 5 AND destination = 'vacation@company.com';
```

Remove URL from Alert Group

sql

-- Completely remove

```
DELETE FROM url_alert_groups
WHERE url_id = 1 AND group_id = 5;
```

-- Or just disable

```
UPDATE url_alert_groups
SET is_active = FALSE
WHERE url_id = 1 AND group_id = 5;
```

Useful Queries

View All Alert Groups

sql

```
SELECT * FROM alert_group_summary;
```

View URL Status with Alert Groups

sql

```
SELECT * FROM url_status_summary;
```

See Who Gets Alerts for a URL

sql

SELECT

```
u.name as url_name,  
ag.group_name,  
agm.notification_type,  
agm.destination,  
agm.member_name
```

FROM monitored_urls u

JOIN url_alert_groups uag ON u.id = uag.url_id

JOIN alert_groups ag ON uag.group_id = ag.id

JOIN alert_group_members agm ON ag.id = agm.group_id

WHERE u.id = 1 AND uag.is_active = TRUE AND ag.is_active = TRUE AND agm.is_active = TR



View Recent Alerts by Group

sql

SELECT

```
ag.group_name,  
u.name as url_name,  
ah.alert_type,  
ah.notification_type,  
ah.destination,  
ah.sent_at
```

FROM alert_history ah

JOIN alert_groups ag ON ah.group_id = ag.id

JOIN monitored_urls u ON ah.url_id = u.id

WHERE ah.sent_at > DATE_SUB(NOW(), INTERVAL 24 HOUR)

ORDER BY ah.sent_at DESC;

Alert Group Best Practices

1. Group Organization

- **critical-ops:** 24/7 monitoring team for critical systems
- **dev-team:** Development team for their applications
- **business-hours:** Support team for non-critical issues
- **executives:** High-level alerts for major outages
- **security-team:** Security-related monitoring alerts

2. Notification Types by Group

- **Critical alerts:** Email + SMS for immediate attention
- **Development alerts:** Email only during business hours
- **Executive alerts:** Email for status updates, no recovery spam

3. Alert Type Configuration

```
sql
```

```
-- Executives: Only down and error alerts, no recovery notifications
```

```
INSERT INTO url_alert_groups (url_id, group_id, alert_on_down, alert_on_up, alert_on_t.  
VALUES (critical_url_id, executives_group_id, TRUE, FALSE, FALSE, TRUE);
```

```
-- Dev team: All alert types for their applications
```

```
INSERT INTO url_alert_groups (url_id, group_id, alert_on_down, alert_on_up, alert_on_t.  
VALUES (dev_url_id, dev_team_group_id, TRUE, TRUE, TRUE, TRUE);
```

4. SMS Configuration

For SMS alerts via email-to-SMS gateways:

- **Verizon:** 1234567890@vztext.com
- **AT&T:** 1234567890@txt.att.net
- **T-Mobile:** 1234567890@tmomail.net
- **Sprint:** 1234567890@messaging.sprintpcs.com

Configuration Management

Update Multiple URLs at Once

```
sql
```

```
-- Add all URLs with 'api' in name to API team alert group
```

```
INSERT INTO url_alert_groups (url_id, group_id)  
SELECT id, 6 FROM monitored_urls WHERE name LIKE '%api%';
```

Bulk Update Group Members

sql

-- Update all dev team emails to new domain

```
UPDATE alert_group_members
SET destination = REPLACE(destination, '@oldcompany.com', '@newcompany.com')
WHERE group_id = (SELECT id FROM alert_groups WHERE group_name = 'dev-team');
```

Emergency Alert Disable

sql

-- Temporarily disable all alerts for maintenance

```
UPDATE alert_groups SET is_active = FALSE;
```

-- Re-enable after maintenance

```
UPDATE alert_groups SET is_active = TRUE;
```

Monitoring the System

Dashboard Query

sql

```
SELECT
    'System Health' as metric,
    COUNT(CASE WHEN last_status = 'up' THEN 1 END) as sites_up,
    COUNT(CASE WHEN last_status = 'down' THEN 1 END) as sites_down,
    COUNT(CASE WHEN last_status = 'error' THEN 1 END) as sites_error,
    COUNT(*) as total_sites
FROM monitored_urls WHERE is_active = TRUE;
```

Alert Volume by Group

sql

```
SELECT
    ag.group_name,
    COUNT(*) as alerts_sent_24h
FROM alert_history ah
JOIN alert_groups ag ON ah.group_id = ag.id
WHERE ah.sent_at > DATE_SUB(NOW(), INTERVAL 24 HOUR)
GROUP BY ag.group_name
ORDER BY alerts_sent_24h DESC;
```

Troubleshooting

No Alerts Being Sent

1. Check if alert groups are active: `SELECT * FROM alert_groups WHERE is_active = FALSE;`
2. Check if group members are active: `SELECT * FROM alert_group_members WHERE is_active = FALSE;`
3. Check URL-group links: `SELECT * FROM url_alert_groups WHERE is_active = FALSE;`
4. Verify SMTP configuration in n8n credentials

Too Many Alerts

1. Check failure thresholds: `SELECT url, failure_threshold FROM monitored_urls;`
2. Review alert cooldown: `SELECT config_value FROM config WHERE config_key = 'notification_cooldown_minutes';`
3. Adjust alert type settings in `url_alert_groups`

Missing Alert Types

Verify alert type configuration:

```
sql

SELECT
    u.name,
    uag.alert_on_down,
    uag.alert_on_up,
    uag.alert_on_timeout,
    uag.alert_on_error
FROM url_alert_groups uag
JOIN monitored_urls u ON uag.url_id = u.id
WHERE uag.url_id = your_url_id;
```

This alert groups system provides the flexibility you need while maintaining simple management of your notification infrastructure!